

0.1 Tiền xử lý văn bản

Đây là bước quan trọng để loại bỏ nhiễu và chuẩn hóa dữ liệu đầu vào.

0.1.1 Tách từ (tokenization)

Tách từ là quá trình chia một chuỗi văn bản thành các đơn vị nhỏ hơn có ý nghĩa, gọi là "token". Token thường là từ, nhưng cũng có thể là số, dấu câu, hoặc các đơn vị có ý nghĩa khác tùy thuộc vào mục đích. Bước này là bước đầu tiên và cơ bản nhất trong hầu hết các tác vụ xử lý ngôn ngữ tự nhiên. Nếu tách từ không chính xác, các bước xử lý sau sẽ bị ảnh hưởng. Có một số phương pháp tách từ như:

- Tách theo khoảng trắng: Đơn giản nhất, chia chuỗi theo dấu cách. Đây là cách đơn giản, dễ thực hiện nhưng có nhược điểm khi không xử lý được nhiều trường hợp phức tạp như: dấu câu dính liền từ, từ ghép
- Tách theo dấu câu: Chia chuỗi theo các dấu câu
- Tách dựa trên từ điển: Sử dụng một từ điển các từ đã biết để nhận diện và tách các từ. Phương pháp này đơn giản, dễ cài đặt nhưng vẫn có các nhược điểm như phụ thuộc vào từ điển, độ chính xác không cao, không giải quyết được trường hợp
- Tách dựa trên quy tắc: Áp dụng các quy tắc ngữ pháp hoặc hình thái học để tách từ (ví dụ: tách tiền tố, hậu tố)
- Tách dựa trên học máy: Sử dụng các mô hình học máy để dự đoán ranh giới từ.

Với từ tiếng Việt, có một số khó khăn khi tách từ như: từ ghép, từ viết tắt, dấu câu dính liền với từ, số đi kèm với đơn vị đo (ví dụ: 10kg, 50km/h). Trong bài toán này nhóm em sử dụng công cụ hỗ trợ trong Python là thư viện `underthesea`

0.1.2 Làm sạch văn bản

Làm sạch văn bản để loại bỏ các yếu tố không mong muốn hoặc không mang ý nghĩa ngữ nghĩa từ văn bản. Một số loại nhiễu thường gặp như:

- HTML tags, XML tags , ví dụ: `<p>`, `<xml>`
- đường dẫn URL
- Ký tự đặc biệt, biểu tượng cảm xúc (emojis)
- Số (trong một số trường hợp cụ thể)
- Khoảng trắng thừa
- Ký tự không phải ASCII (nếu cần thiết)

Tuỳ trong một số bài toán cụ thể thì có thể linh hoạt xác định và loại bỏ nhiều

0.1.3 Loại bỏ ký tự đặc biệt và từ dừng

Từ dừng là các từ rất phổ biến trong ngôn ngữ nhưng thường không mang nhiều ý nghĩa phân biệt hoặc ngữ nghĩa quan trọng cho các tác vụ xử lý ngôn ngữ tự nhiên, ví dụ: “là”, “và”, “của”, ... Việc loại bỏ các từ dừng giúp giảm kích thước dữ liệu, thời gian xử lý và cải thiện hiệu suất của mô hình. Ở đây nhóm sử dụng một từ điển từ dừng tự định nghĩa để lọc các từ dừng. Về các ký tự đặc biệt, việc loại bỏ những ký tự này cũng để giảm nhiễu. Tuy nhiên cần cân nhắc ngữ cảnh. Ví dụ: trong phân tích cảm xúc, dấu chấm than ! có thể mang ý nghĩa tích cực mạnh

0.1.4 Chỉnh sửa từ

Trong văn bản có thể có một số lỗi chính tả, lỗi gõ phím hoặc các biến thể không chuẩn của từ. Do đó cần phải sửa để đưa về dạng chuẩn để đảm bảo tính nhất quán của dữ liệu. Lỗi chính tả có thể tạo ra nhiều "từ" khác nhau cho cùng một khái niệm, làm giảm hiệu quả của các mô hình. Có một số phương pháp để chỉnh sửa từ như:

- Dựa trên từ điển: So sánh từ với một từ điển chuẩn, tìm từ gần nhất (ví dụ: khoảng cách Levenshtein).
- Dựa trên mô hình ngôn ngữ: Sử dụng các mô hình thống kê hoặc học sâu để dự đoán từ đúng dựa trên ngữ cảnh.
- Áp dụng các quy tắc để sửa các lỗi phổ biến.

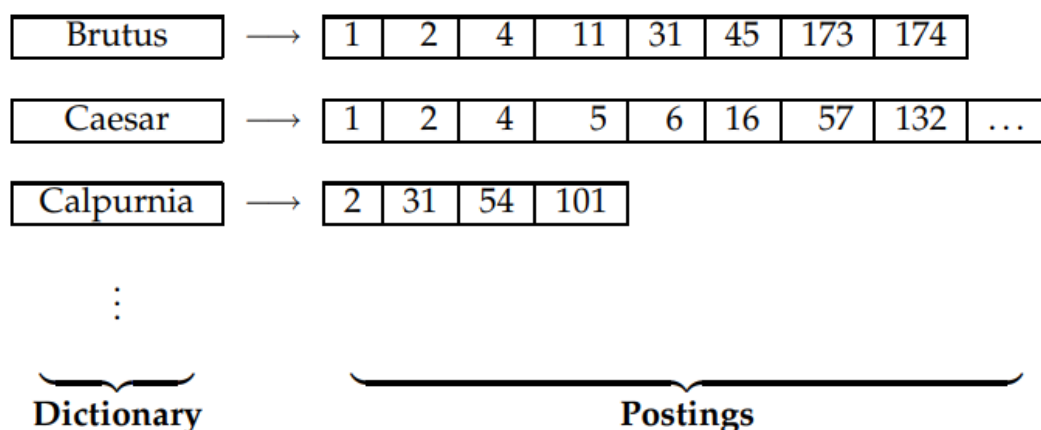
Trong bài này sử dụng thư viện `underthesea` hỗ trợ chỉnh sửa từ.

0.2 Chỉ mục

Sau khi tiến hành tiền xử lý để tách từ thì sẽ đến bước xây dựng chỉ mục từ từ điển chứa các từ đã được tách.

0.2.1 Chỉ mục ngược

Chỉ mục ngược là chỉ mục mà ánh xạ ngược từ từ (term) đến tài liệu mà từ đó xuất hiện. Ý tưởng chính của chỉ mục ngược là giữ một tập từ điển các từ (thường được gọi là tập từ vựng). Với mỗi từ, xây dựng một danh sách chứa các tài liệu từ đó xuất hiện. Mỗi phần tử trong danh sách còn được gọi là posting. Danh sách chứa các posting gọi là postings list **manning2008introduction** và tập hợp các postings list được gọi là postings. Ví dụ về chỉ mục ngược và các thành phần được mô tả như trên hình 0.1.



Hình 0.1: Một ví dụ về chỉ mục ngược và các thành phần của nó

0.2.2 Các bước lập chỉ mục

1. Tạo chuỗi các cặp (token đã chỉnh sửa, ID tài liệu) từ văn bản gốc
2. Sắp xếp danh sách theo thuật ngữ, sau đó theo ID tài liệu (đây là bước cốt lõi của việc lập chỉ mục)
3. Gộp các mục trùng lặp của một từ (thuật ngữ) trong cùng một tài liệu
4. Chỉ mục được tách thành từ điển và postings
5. Bổ sung thông tin tần suất của tài liệu (phục vụ cho việc xây dựng mô hình không gian vector và BM25 sau này)

0.2.3 Chỉ mục vị trí

Chỉ mục vị trí được xây dựng như một bản nâng cấp của chỉ mục ngược do đó cấu trúc tương tự như chỉ mục ngược, khác biệt ở chỗ trong danh sách postings (postings lists), lưu trữ vị trí xuất hiện của mỗi thuật ngữ trong tài liệu. Ví dụ về một chỉ mục vị trí:

angels: 2: {36,174,252,651}; 4: {12,22,102,432}; 7: {17};

Trong đó, 2 là ID của tài liệu và danh sách bên trong chứa thứ tự vị trí của từ đó xuất hiện trong văn bản.

0.3 Các thang đo chấm điểm mức độ liên quan của tài liệu

0.3.1 TF-IDF

TF-IDF là phương pháp thống kê nhằm đánh giá tầm quan trọng của một từ đối với 1 tài liệu trong 1 tập hợp tài liệu dựa trên hai yếu tố:

- Term Frequency (TF): Tần suất xuất hiện của từ trong văn bản.
- Inverse Document Frequency (IDF): Mức độ hiếm của từ đó trong toàn bộ tập

tài liệu. Từ càng hiếm thì IDF càng cao.

Một từ quan trọng là từ xuất hiện nhiều trong 1 tài liệu cụ thể (TF cao) nhưng lại ít xuất hiện trong các tài liệu khác (IDF cao)

Các công thức của TF-IDF như sau:

$$TF(t, d) = \frac{\text{Số lần xuất hiện của từ } t \text{ trong văn bản } d}{\text{Tổng số từ trong văn bản } d}$$

$$IDF(t, D) = \log \left(\frac{\text{Tổng số văn bản}}{\text{Tổng số văn bản có chứa từ } t} \right)$$

$$TF - IDF(t, d, D) = TF(t, d) \times IDF(t, D)$$

Từ công thức này có thể chấm điểm độ liên quan với một truy vấn

0.3.2 BM25

BM25 là một biến thể nâng cao của mô hình TF-IDF, thuộc họ các mô hình truy hồi thông tin dựa trên xác suất. Công thức phổ biến nhất để tính điểm BM25 của một tài liệu D đối với truy vấn Q (chứa các từ khóa $q_1, q_2, \dots, q_n$) là:

$$\text{score}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot (1 - b + b \cdot \frac{|D|}{\text{avgdl}})}$$

Trong đó:

- $f(q_i, D)$: Tần suất xuất hiện của từ khóa q_i trong tài liệu D .
- $|D|$: Độ dài của tài liệu D (tổng số từ).
- avgdl: Độ dài trung bình của tất cả các tài liệu trong tập dữ liệu.
- k_1 : Một tham số tự do (thường chọn từ 1.2 đến 2.0). Nó kiểm soát mức độ ảnh hưởng của tần suất từ khóa (TF).
- b : Một tham số tự do (thường là 0.75). Nó quyết định mức độ ảnh hưởng của độ dài tài liệu, chuẩn hóa theo độ dài tài liệu.
- $\text{IDF}(q_i)$: Trọng số nghịch đảo của tần suất tài liệu, thường được tính bằng:

$$\text{IDF}(q_i) = \ln \left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1 \right)$$

(với N là tổng số tài liệu và $n(q_i)$ là số tài liệu chứa từ q_i).

Giải thích 2 tham số ảnh hưởng k_1 và b .

1. **TF saturation:** Khi TF tăng, điểm số tăng nhưng với tốc độ giảm dần (di-

minishing returns). Được kiểm soát bởi k_1 .

- $k_1 = 0$: TF không ảnh hưởng
- $k_1 \rightarrow \infty$: TF ảnh hưởng tuyến tính

2. **Length normalization**: Chuẩn hóa theo độ dài document, được kiểm soát bởi b .

- $b = 0$: Không chuẩn hóa theo độ dài
- $b = 1$: Chuẩn hóa hoàn toàn theo độ dài

3. **IDF**: Term hiếm có IDF cao $\rightarrow$ đóng góp lớn hơn vào điểm số.

0.4 Các thước đo đánh giá

Với cách đánh giá dựa trên thứ hạng, đề tài sử dụng một số thước đo như sau:

0.4.1 Precision@K (Precision at K)

Precision at k là tỷ lệ giữa số lượng tài liệu liên quan được tìm thấy trong k kết quả đầu tiên chia cho giá trị k . Chỉ số này trả lời cho câu hỏi: "Trong số k kết quả mà hệ thống đề xuất, có bao nhiêu phần trăm thực sự hữu ích?"

Công thức: $\text{Precision}@k = \frac{\#\{\text{tài liệu liên quan trong } k \text{ kết quả đầu tiên}\}}{k}$

Đặc điểm của thước đo:

- Chỉ quan tâm đến tập kết quả hàng đầu, không quan tâm đến các tài liệu liên quan nằm ngoài vị trí k .
- Dễ hiểu và trực quan đối với người dùng cuối (ví dụ: trang đầu tiên của Google thường có $k = 10$).

Nhược điểm của thước đo là không tính đến thứ tự của các tài liệu liên quan bên trong tập k (ví dụ: tài liệu liên quan nằm ở vị trí thứ 1 hay thứ 10 thì $P@10$ vẫn như nhau).

0.4.2 Recall at k (R@k)

Recall at k là tỷ lệ giữa số lượng tài liệu liên quan được tìm thấy trong k kết quả đầu tiên chia cho tổng số tài liệu liên quan thực tế có trong toàn bộ cơ sở dữ liệu. Nó trả lời cho câu hỏi: "Hệ thống đã tìm thấy được bao nhiêu phần trăm tài liệu tốt trong tổng số tài liệu tốt hiện có chỉ với k kết quả đầu?". Công thức của Recall@K:

$$\text{Recall}@k = \frac{\#\{\text{tài liệu liên quan được tìm thấy trong } k \text{ kết quả đầu tiên}\}}{\#\{\text{tổng số tài liệu liên quan trong toàn bộ cơ sở dữ liệu}\}}$$

$R@k$ luôn tăng hoặc giữ nguyên khi k tăng. Chỉ số này rất quan trọng trong các hệ thống mà việc bỏ sót thông tin là nghiêm trọng (ví dụ: tìm kiếm hồ sơ y tế hoặc bằng sáng chế).

0.4.3 F1 at k (F1@k)

F1 at k là trung bình điều hòa giữa $Precision@k$ và $Recall@k$. Nó giúp cân bằng giữa việc tìm đúng (Precision) và tìm đủ (Recall) trong phạm vi k kết quả đầu tiên.

$$F1@k = 2 \cdot \frac{P@k \cdot R@k}{P@k + R@k}$$

0.4.4 Mean Reciprocal Rank (MRR)

MRR là giá trị trung bình của các Reciprocal Rank (nghịch đảo của thứ hạng) trên một tập hợp các truy vấn hoặc các mốc thời gian.

Khác với $Precision@k$ (chỉ quan tâm đến số lượng trong top k), MRR tập trung vào vị trí của tài liệu liên quan đầu tiên. Nếu hệ thống đưa tài liệu đúng lên vị trí số 1, nó sẽ nhận điểm tối đa. Nếu tài liệu đúng nằm càng xa vị trí đầu tiên, điểm số sẽ giảm dần theo hàm nghịch đảo.

Công thức tính toán như sau: Trước hết ta tính Reciprocal Rank (RR) cho từng truy vấn i :

$$RR_i = \frac{1}{rank_i}$$

Trong đó $rank_i$ là vị trí của tài liệu liên quan đầu tiên được tìm thấy cho truy vấn đó. Nếu không có tài liệu liên quan nào được tìm thấy, $RR_i = 0$.

Mean Reciprocal Rank (MRR) được tính bằng trung bình cộng của các RR_i trên tổng số N truy vấn:

$$MRR = \frac{1}{N} \sum_{i=1}^N \frac{1}{rank_i}$$

Một số đặc điểm của thước đo này:

- Ưu tiên vị trí dẫn đầu: MRR cực kỳ nhạy bén với việc tài liệu đúng nằm ở vị trí thứ 1 hay thứ 2. Ví dụ: vị trí thứ 1 cho 1 điểm, nhưng vị trí thứ 2 chỉ còn 0.5 điểm.
- Phù hợp với tìm kiếm thực tế: Rất phù hợp cho các hệ thống mà người dùng chỉ cần tìm thấy một kết quả đúng duy nhất (như tìm kiếm câu trả lời cho một câu hỏi cụ thể hoặc chọn một cặp tiền tệ tốt nhất để giao dịch).
- Hạn chế: MRR chỉ quan tâm đến tài liệu liên quan đầu tiên. Nếu một danh sách có nhiều tài liệu liên quan khác ở phía sau, MRR sẽ hoàn toàn bỏ qua

chúng.

0.5 Elasticsearch

Elasticsearch là một công cụ dựa trên phần mềm Lucene - phần mềm tìm kiếm và truy xuất thông tin với hơn 15 năm kinh nghiệm về truy vấn toàn bộ văn bản và tìm kiếm. Nó cung cấp một bộ máy tìm kiếm dạng phân tán, có đầy đủ công cụ với một giao diện web HTTP có hỗ trợ dữ liệu JSON. Elasticsearch được phát triển bằng Java và được phát hành dạng mã nguồn mở theo giấy phép Apache. Elasticsearch là một CSDL kiểu NoSQL với nhiệm vụ chính là lưu và truy vấn tài liệu. Trong Elasticsearch, tất cả các tài liệu được hiển thị dưới dạng JSON.

0.5.1 Các thành phần chính

a, Tài liệu (Document)

Một tài liệu là bản ghi thông tin nhỏ nhất được lưu bởi Elasticsearch và được biểu diễn dưới dạng JSON. Một tài liệu bao gồm nhiều trường cặp khóa-giá trị (key-value) với các kiểu khác nhau và có thể có schema xác định trước hoặc không có schema, suy đoán kiểu dữ liệu của các trường mỗi khi chúng được lập chỉ mục.

b, Chỉ mục (Index)

Một chỉ mục là một tập hợp logic của các document có cùng schema, được xác định bằng một tên chỉ mục.

c, Shard

Các chỉ mục của Elasticsearch được chia thành các đơn vị có thể quản lý gọi là shard, là tập hợp các document. Shard là đơn vị cơ bản của tìm kiếm và được sao chép trên nhiều node để dự phòng và chịu lỗi.

d, Node

Một node là một máy ảo(instance) độc lập của Elasticsearch và quản lý một tập hợp các shard thuộc về một hoặc nhiều chỉ mục. Node có thể có các vai trò khác nhau như data node, master node và ingest node.

e, Cụm(Cluster)

Một cụm trong Elasticsearch là một tập hợp các node được kết nối với nhau. Tất cả node trong một cụm có thể xử lý yêu cầu từ client và giao tiếp với nhau. Mỗi node trong cụm sở hữu một phần shard thuộc về một chỉ mục.

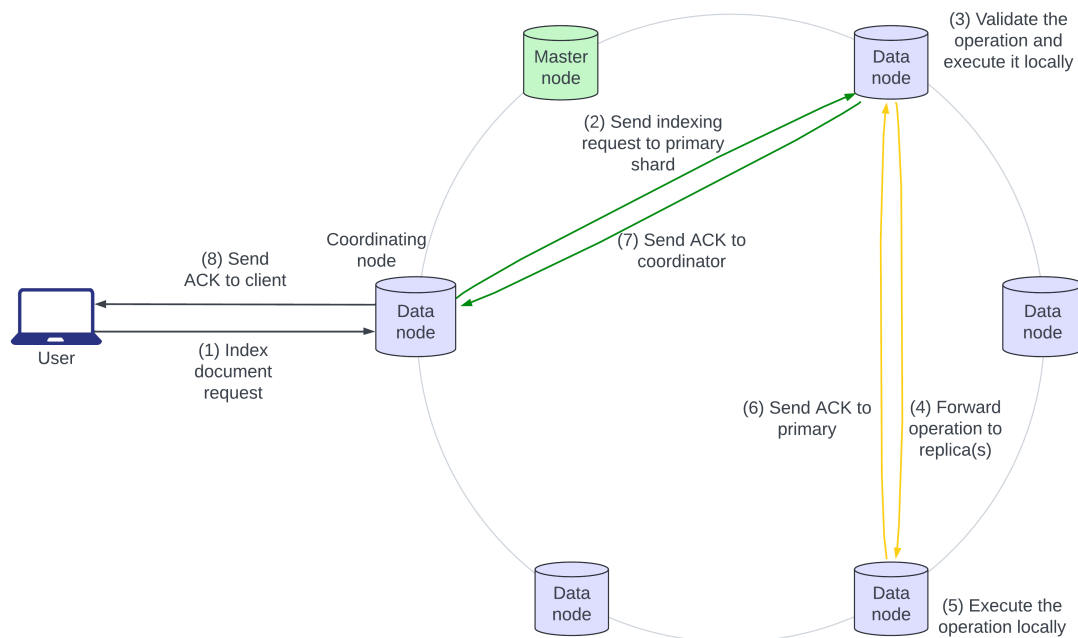
0.5.2 Cách Elasticsearch đánh chỉ mục (index)

Elasticsearch sử dụng một chỉ mục ngược (inverted index) để đánh chỉ mục. Các bước như sau:

1. Người dùng gửi một tài liệu dưới dạng JSON để Elasticsearch lập chỉ mục.

Nếu tài liệu đã tồn tại, các trường mới được thêm vào và các trường hiện có bị ghi đè. Node nhận yêu cầu đầu tiên là "node điều phối".

2. Node điều phối xác định primary shard của tài liệu đến, thường dựa trên ID của document, và chuyển tiếp yêu cầu tới data node sở hữu primary shard đó.
3. Primary shard xác thực thao tác và thực thi nó tại chỗ.
4. Primary shard sau đó chuyển tiếp thao tác tới tất cả replica của nó một cách song song.
5. Các replica shard áp dụng thao tác tại chỗ trên các node của chúng.
6. Các bước 6, 7 và 8 thể hiện việc xác nhận ghi (acknowledgment) bắt đầu từ replica shard lên primary shard, tới node điều phối, và tới node yêu cầu.



Hình 0.2: Các bước lập chỉ mục trong Elasticsearch

0.5.3 Cách Elasticsearch truy vấn

Elasticsearch sử dụng BM25 làm thước đo mặc định đánh giá xếp hạng mức độ liên quan của truy vấn với tài liệu. Ngoài ra thì TF-IDF cũng có thể được sử dụng nếu được chỉ định. Elasticsearch hỗ trợ các loại truy vấn như:

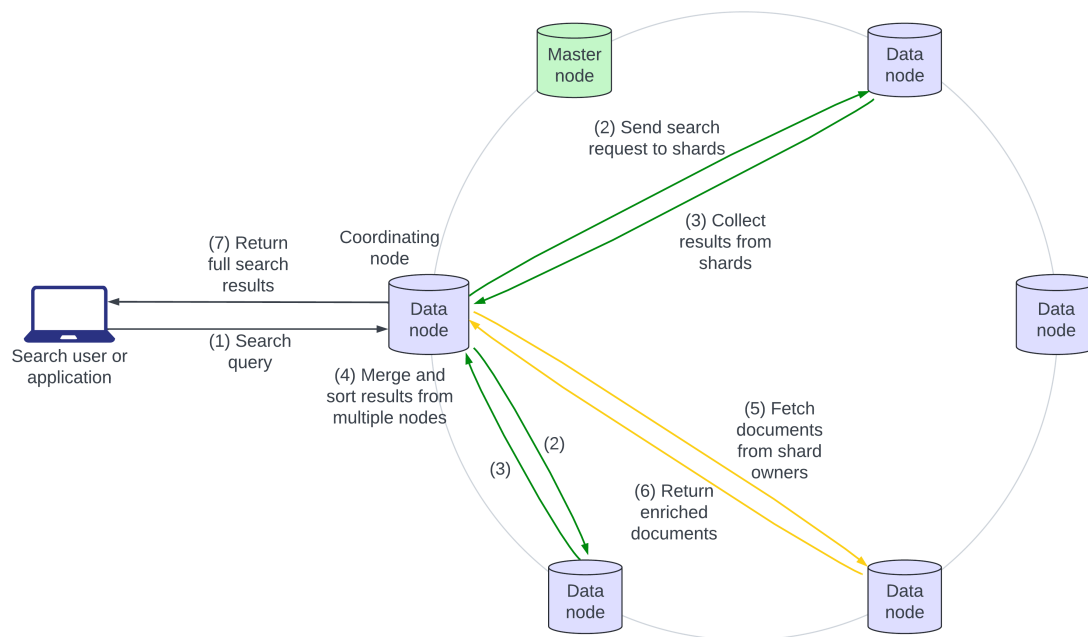
- Match Query: Sử dụng tìm kiếm toàn văn với phân tích token.
- Term Query: Tìm các kết quả khớp chính xác mà không qua xử lý văn bản.
- Bool Query: Kết hợp nhiều truy vấn bằng các mệnh đề must, should và must_not.

Quá trình thực hiện truy vấn, tìm kiếm gồm các pha như sau:

- Pha truy vấn: Yêu cầu được phát tới các shard liên quan.
- Pha tính điểm: Mỗi shard tính điểm liên quan của tài liệu.
- Pha tổng hợp: Kết quả được hợp nhất và xếp hạng trước khi trả về phản hồi cuối cùng.

Các bước truy vấn trong một cụm Elasticsearch như sau:

1. Người dùng hoặc ứng dụng gửi một truy vấn tìm kiếm. Truy vấn có thể được xử lý bởi bất kỳ node nào trong cụm. Node xử lý yêu cầu được gọi là "node điều phối".
2. Node điều phối phát broadcast truy vấn tới tất cả các shard liên quan và các replica của chúng.
3. Mỗi shard thực thi truy vấn cục bộ và trả về một tập kết quả lightweight cho node điều phối.
4. Node điều phối hợp nhất các kết quả nhận được. Đây là kết thúc pha "truy vấn". Pha truy vấn xác định các tài liệu sơ bộ tạo thành kết quả tìm kiếm, nhưng tài liệu đầy đủ vẫn cần được truy xuất.
5. Node điều phối gửi yêu cầu truy xuất tới các shard sở hữu, những shard này bổ sung thêm các tài liệu trong tập kết quả.
6. Các tài liệu đã được bổ sung được trả về cho node điều phối.
7. Tập kết quả tìm kiếm đầy đủ, đã được xếp hạng và đã được bổ sung, được trả về cho người gọi.



Hình 0.3: Các bước truy vấn trong Elasticsearch